BTS SIO – OPTION SISR
1ÈRE ANNÉE – DOSSIER DE RÉALISATION

ADMINISTRATION LINUX :

Utilisateurs, groupes, permissions & sudo

Sécurisation des accès d'un serveur Ubuntu Server

BECKY
Serveur : serv02

17/11/2025

INDEX

Sommaire

Partie 1 – Contexte et cadrage du projet 3

Partie 2 – Exploration du système et des fichiers de configuration . . . 5

Partie 3 – Gestion avancée des comptes utilisateurs 11

Partie 4 – Gestion des groupes et organisation du système 15

Partie 5 – Permissions, modes spéciaux et sécurité 18

Partie 6 – Gestion des privilèges avec sudo et traçabilité 21

Partie 7 – Tests, résultats et bilan du projet 25

Annexe – Récapitulatif des commandes 28

PARTIE 1 :

Contexte et cadrage du projet

1) Mise en situation

Un serveur Linux, nommé **serv02**, doit être mis à disposition d'une petite équipe de développement et accueillir des stagiaires pendant quelques semaines. En tant que technicienne systèmes et réseaux, je dois préparer cet environnement : créer les comptes, les organiser en groupes, ouvrir un espace de travail commun, déléguer certaines tâches d'administration sans donner un accès complet au système, et conserver une trace des actions sensibles.

Cette mise en situation est un scénario pédagogique : elle donne un cadre professionnel au TP « Gestion des utilisateurs, groupes et permissions » (niveau BTS) et permet de présenter les manipulations comme une véritable réalisation d'administration système.

2) Problématique

> **QUESTION POSÉE**
> Comment organiser les comptes, les groupes et les droits d'accès d'un serveur Linux pour que chaque personne dispose uniquement des accès nécessaires à son travail (**principe du moindre privilège**), tout en gardant la maîtrise et la **traçabilité** des actions d'administration ?

3) Objectifs du projet

- Comprendre le rôle des fichiers de configuration `/etc/passwd`, `/etc/shadow` et `/etc/group`.
- Gérer le cycle de vie d'un compte : création, modification, verrouillage, expiration, suppression.
- Structurer les utilisateurs en groupes et mettre en place un dossier de travail partagé.
- Maîtriser les permissions classiques et les modes spéciaux (SetUID, SetGID, Sticky Bit).
- Déléguer des droits d'administration précis avec `sudo` et vérifier leur traçabilité dans les journaux.

4) Environnement technique

Élément	Détail
Serveur	serv02 – Linux Ubuntu Server (famille Debian), accès en console
Compte administrateur	**dev** – UID 1000, membre du groupe **sudo**
Comptes créés	stagiaire1 (renommé **stagiaire2** en cours de projet), stagiaire3, stagiaire4, stagiaire5
Groupes créés	devs (supprimé en partie 4), projets (renommé **projets_dev**, GID 2000), adminlocal
Dossiers et fichiers	`/srv/dev`, `/home/projet` (avec notes.txt), `/tmp/partage`, script `/usr/local/bin/helloroot.sh`
Outils utilisés	useradd, usermod, passwd, chage, groupadd, groupmod, groupdel, gpasswd, chmod, chown, chgrp, visudo, sudo, id, getent
Preuves	Captures d'écran du terminal et journal `/var/log/auth.log`

5) Démarche suivie

Le projet est découpé en cinq phases progressives. Chaque partie suit la même logique : objectif, étapes réalisées avec leurs captures, difficultés rencontrées puis résultat obtenu.

Partie	Contenu	Résultat attendu
2 – Exploration	Lecture et analyse des fichiers d'identité du système	Comprendre où et comment Linux stocke comptes et groupes
3 – Comptes	Création, modification, expiration, verrouillage	Un compte stagiaire correctement paramétré
4 – Groupes	Création, renommage, suppression, dossier partagé	Un espace de travail réservé à un groupe
5 – Permissions	chmod 750, SetGID, Sticky Bit, SetUID	Droits maîtrisés, modes spéciaux expliqués
6 – sudo	Règles sudoers, groupe adminlocal, auth.log	Délégation contrôlée et traçable
7 – Tests et bilan	Tableau de validation, compétences, conclusion	Vision critique et réutilisable du projet

> **CONVENTION DE LECTURE**
> Les commandes sont écrites dans des blocs sombres. Chaque figure est une capture réalisée sur serv02 pendant le TP (17/11/2025) : elle sert de preuve de réalisation. Le hash du mot de passe visible dans `/etc/shadow` a été masqué pour la publication.

PARTIE2 :

Exploration du système et des fichiers de configuration

Objectif : comprendre comment Linux identifie les utilisateurs et les groupes avant de les modifier. Un administrateur qui ne sait pas lire ces fichiers ne peut ni diagnostiquer un problème de connexion, ni auditer les comptes d'un serveur.

Étape 1 – Afficher les fichiers fondamentaux

J'ai affiché les trois fichiers avec la commande `cat`. Chaque ligne de `/etc/passwd` suit le format `nom:x:UID:GID:commentaire:répertoire:shell`. Par exemple, mon compte apparaît sous la forme `dev:x:1000:1000:serv02:/home/dev:/bin/bash`.

```
dev@serv02:~$ cat /etc/passwd
```

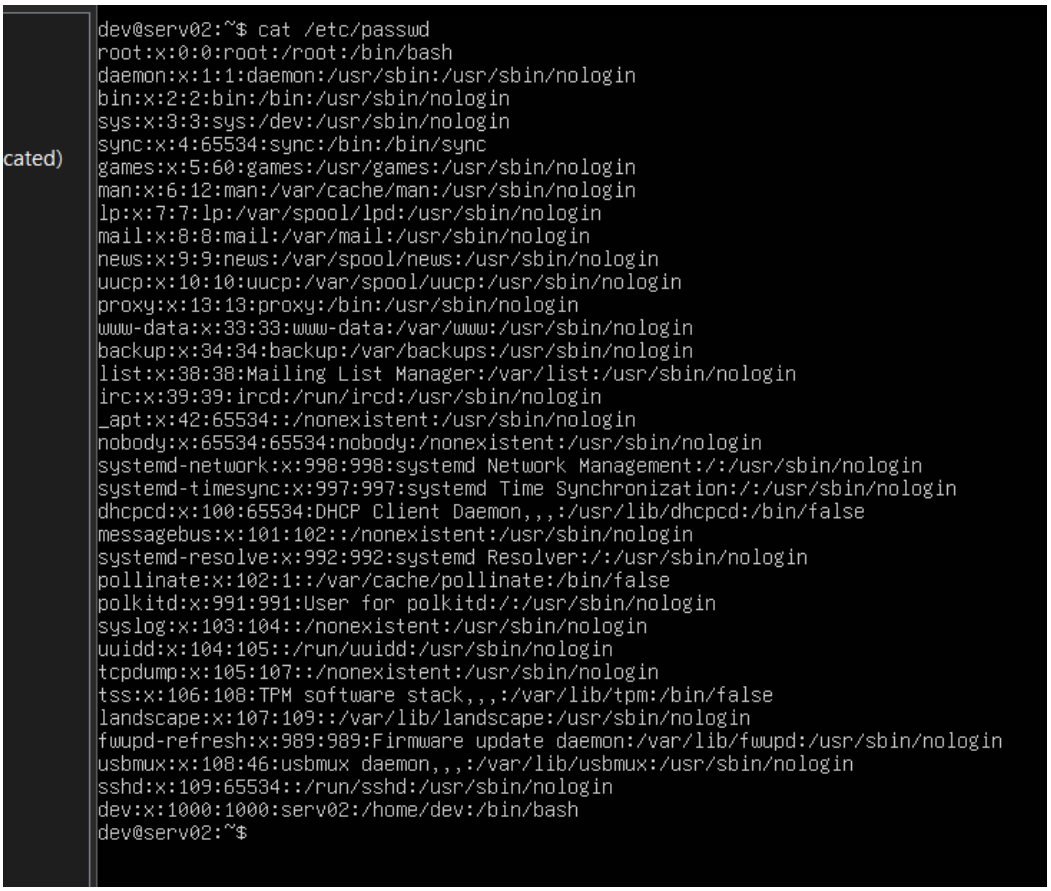


Figure 1 – Contenu de /etc/passwd : un compte par ligne, comptes système (nologin) et compte dev (UID 1000).

Le fichier `/etc/shadow` contient les mots de passe chiffrés. Un utilisateur normal n'y a pas accès : ma première tentative a donc été refusée, et j'ai dû utiliser `sudo`.

```
dev@serv02:~$ cat /etc/shadow           # Permission denied
dev@serv02:~$ sudo cat /etc/shadow
```

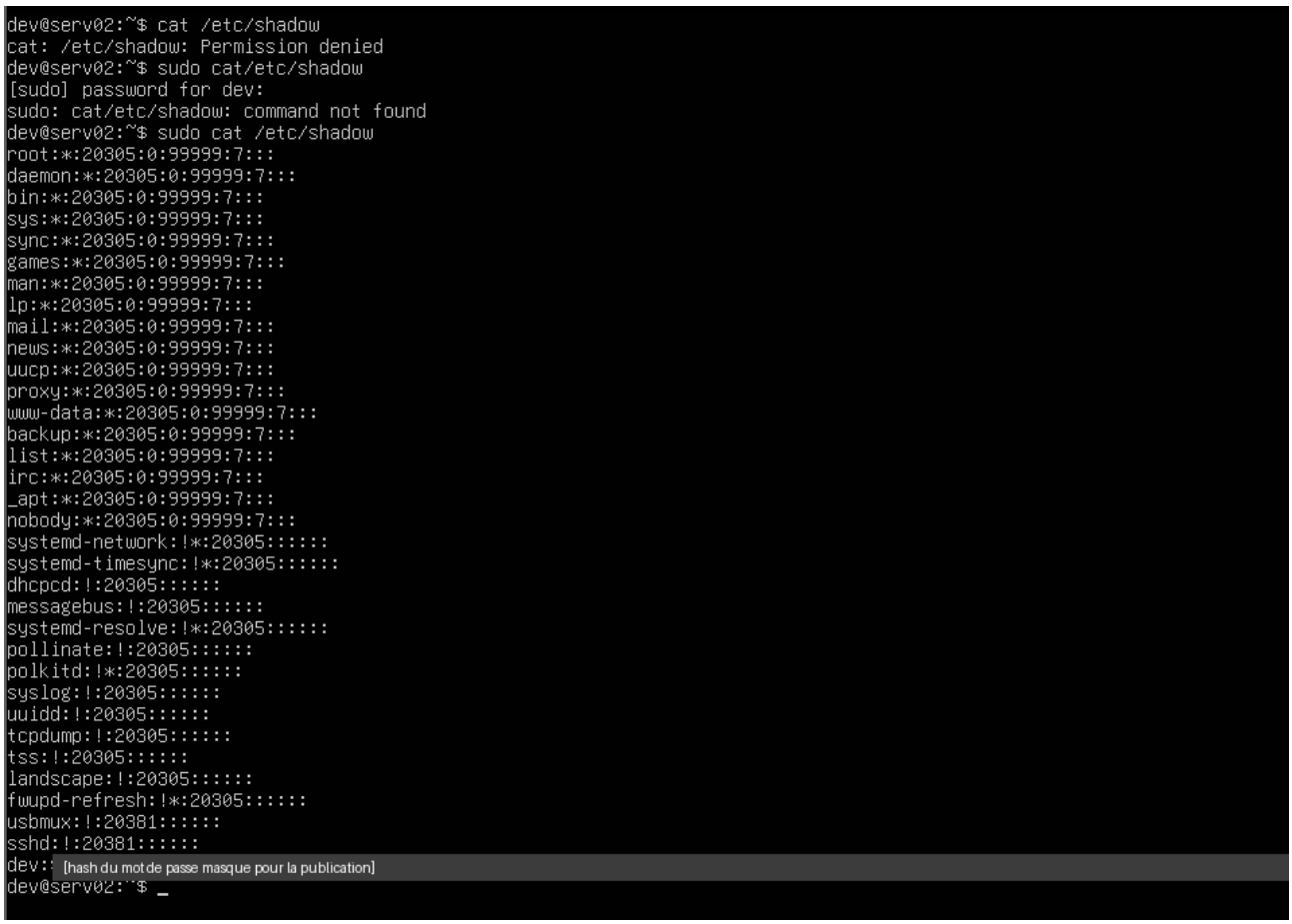


Figure 2 – Accès refusé sans sudo, puis lecture de /etc/shadow avec sudo (hash du compte dev masqué).

> **DIFFICULTÉ RENCONTRÉE**
> Une première commande `sudo cat/etc/shadow` a échoué avec « command not found » : il manquait l'espace entre `cat` et le chemin. Le shell a cherché une commande nommée « cat/etc/shadow ». Une fois la syntaxe corrigée, l'accès a fonctionné.

Dans `/etc/shadow`, les comptes système affichent * ou **!** à la place du hash : aucune connexion par mot de passe n'est possible pour eux, ce qui est voulu. Le hash du compte `dev` commence par **6**, signe d'un chiffrement SHA-512 salé. Le fichier est lisible uniquement par root et le groupe shadow, ce qui empêche un utilisateur de copier les hashs pour les casser hors ligne.

Enfin, `/etc/group` liste les groupes sous la forme `nom:x:GID:membres`. On y voit par exemple `sudo:x:27:dev` : mon compte appartient au groupe qui ouvre l'accès à sudo.

```
dev@serv02:~$ cat /etc/group
```

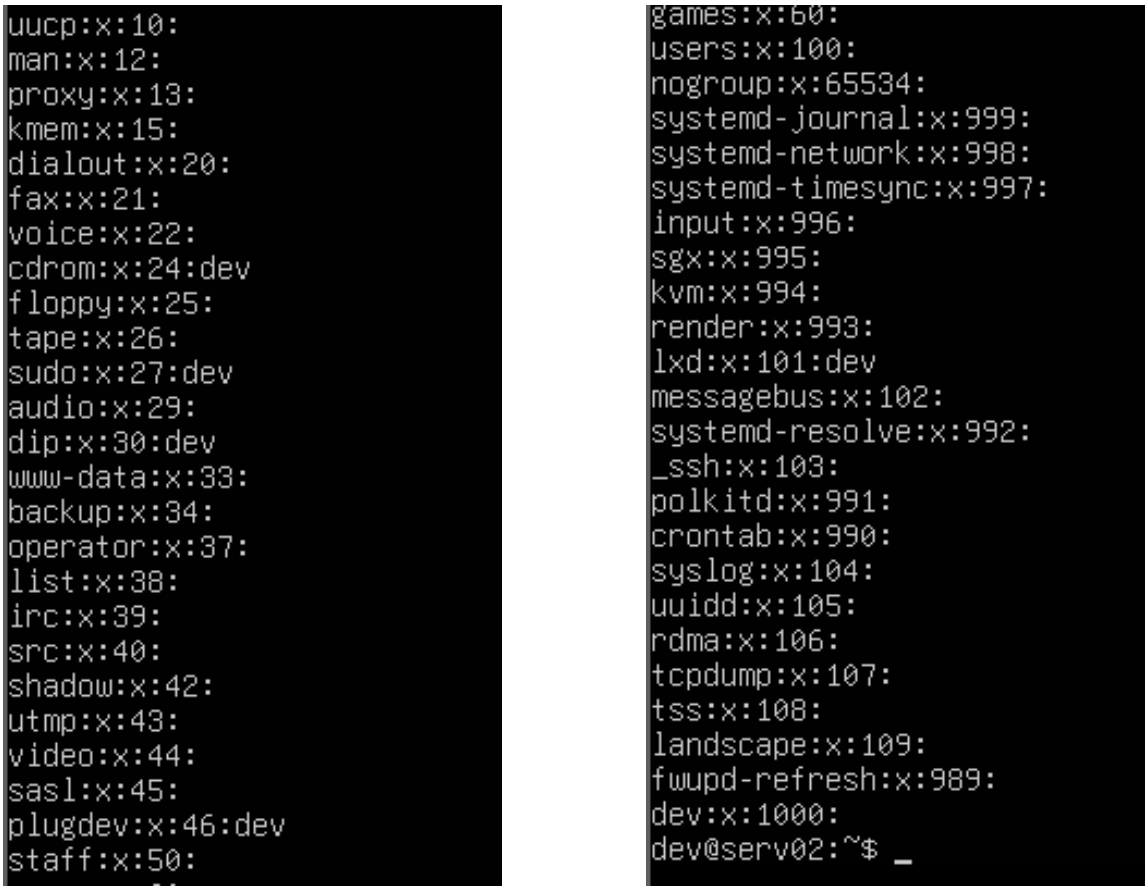


Figure 3 – Contenu de /etc/group (affiché en deux moitiés pour rester lisible).

Étape 2 – Rôle, informations et risques de chaque fichier

Fichier	Rôle dans le système	Informations stockées	Risque en cas de mauvaise manipulation
`/etc/passwd`	Liste tous les comptes ; base de l'identification des utilisateurs.	Nom, UID, GID, commentaire, répertoire personnel, shell par défaut.	Fichier supprimé ou corrompu : plus aucune connexion possible, système inutilisable.
`/etc/shadow`	Stocke les mots de passe chiffrés ; lecture réservée à root et au groupe shadow.	Hash, date de dernier changement, durées minimale/maximale, avertissement, expiration du compte.	Lisible par tous : mots de passe exposés au craquage. Supprimé : plus de connexion, même pour root.
`/etc/group`	Définit les groupes et leurs membres ; permet une gestion collective des droits.	Nom du groupe, GID, liste des membres (exemple : sudo:x:27:dev).	Groupe supprimé : membres sans accès aux ressources du groupe et fichiers au GID orphelin.

Étape 3 – Analyse de mon propre utilisateur

Les commandes `id` et `id -Gn` donnent l'identité complète de mon compte.



Figure 4 – UID et GID du compte dev : 1000 pour les deux.



Figure 5 – Shell de connexion : /bin/bash.

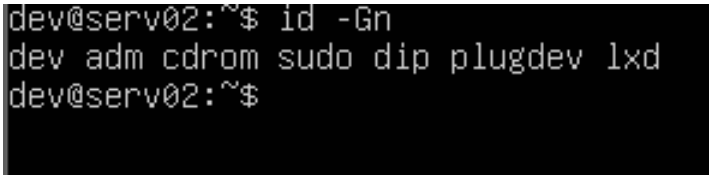


Figure 6 – Commande id -Gn : noms des groupes (dev, adm, cdrom, sudo, dip, plugdev, lxd).



Figure 7 – Commande id : UID, GID et groupes avec leurs numéros.

Information	Valeur relevée
UID / GID	1000 (dev) / 1000 (dev)
Répertoire personnel	`/home/dev`
Shell	`/bin/bash`
Groupes secondaires	adm (4), cdrom (24), sudo (27), dip (30), plugdev (46), lxd (101)

> **POINT DE VIGILANCE SÉCURITÉ**
> Le groupe **sudo** donne le droit d'administrer le serveur. Le groupe **lxd** mérite aussi une attention particulière : ses membres peuvent piloter des conteneurs avec des droits très étendus, ce qui équivaut en pratique à un accès root. Sur un serveur de production, on n'y ajoute que des comptes de confiance.

Étape 4 – Explorer avec getent

La commande `getent` interroge les sources d'identité configurées (fichiers locaux, mais aussi annuaire LDAP ou Active Directory si le serveur y est rattaché). Ici, le résultat est identique à celui de `cat`, car seuls les fichiers locaux sont utilisés.

```
dev@serv02:~$ getent passwd
dev@serv02:~$ getent group
```

```
dev@serv02:~$ getent passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:102::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:992:992:systemd Resolver:/:/usr/sbin/nologin
pollinate:x:102:1::/var/cache/pollinate:/bin/false
polkitd:x:991:991:User for polkitd:/:/usr/sbin/nologin
syslog:x:103:104::/nonexistent:/usr/sbin/nologin
uuidd:x:104:105::/run/uuidd:/usr/sbin/nologin
tcpdump:x:105:107::/nonexistent:/usr/sbin/nologin
tss:x:106:108:TPM software stack,,,:/var/lib/tpm:/bin/false
landscape:x:107:109::/var/lib/landscape:/usr/sbin/nologin
fwupd-refresh:x:989:989:Firmware update daemon:/var/lib/fwupd:/usr/sbin/nologin
usbmux:x:108:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
sshd:x:109:65534::/run/sshd:/usr/sbin/nologin
dev:x:1000:1000:serv02:/home/dev:/bin/bash
dev@serv02:~$
```

Figure 8 – Résultat de getent passwd.

```
uucp:x:10:
man:x:12:
proxy:x:13:
kmem:x:15:
dialout:x:20:
fax:x:21:
voice:x:22:
cdrom:x:24:dev
floppy:x:25:
tape:x:26:
sudo:x:27:dev
audio:x:29:
dip:x:30:dev
www-data:x:33:
backup:x:34:
operator:x:37:
list:x:38:
irc:x:39:
src:x:40:
shadow:x:42:
utmp:x:43:
video:x:44:
sasl:x:45:
plugdev:x:46:dev
```

```
games:x:60:
users:x:100:
nogroup:x:65534:
systemd-journal:x:999:
systemd-network:x:998:
systemd-timesync:x:997:
input:x:996:
sgx:x:995:
kvm:x:994:
render:x:993:
lxd:x:101:dev
messagebus:x:102:
systemd-resolve:x:992:
_ssh:x:103:
polkitd:x:991:
crontab:x:990:
syslog:x:104:
uuidd:x:105:
rdma:x:106:
tcpdump:x:107:
tss:x:108:
landscape:x:109:
fwupd-refresh:x:989:
dev:x:1000:
dev@serv02:~$ _
```

Figure 9 – Résultat de getent group (en deux moitiés).

RÉSULTAT OBTENU

Je sais lire et interpréter les trois fichiers d'identité, identifier mon UID, mes groupes et leur impact (sudo, lxd), et je comprends pourquoi `/etc/shadow` est protégé. Cette base est nécessaire pour la création de comptes qui suit.

PARTIE3 :

Gestion avancée des comptes utilisateurs

Objectif : maîtriser le cycle de vie complet d'un compte, de la création au verrouillage, comme lors de l'arrivée d'un stagiaire.

Étape 1 – Création du compte avec son répertoire personnel

```
dev@serv02:~$ sudo useradd -m stagiaire1
dev@serv02:~$ sudo passwd stagiaire1
```

L'option `-m` crée le répertoire personnel à partir du modèle `/etc/skel`. Sans elle, le compte n'aurait pas de dossier. La commande `passwd` définit ensuite le mot de passe : un compte créé sans mot de passe reste inutilisable (verrouillé).

Étape 2 – Vérifier le home et le shell par défaut

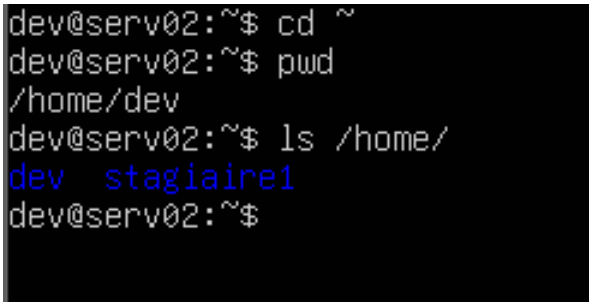


Figure 10 – Le répertoire /home/stagiaire1 existe à côté de /home/dev.



Figure 11 – Ligne du compte dans /etc/passwd : UID 1001, GID 1001, shell par défaut /bin/sh.

Le shell par défaut attribué par `useradd` est `/bin/sh`, un shell minimal sans confort d'utilisation (pas de complétion ni d'historique). Pour un utilisateur interactif, on préfère `/bin/bash`, d'où l'étape suivante.

Étape 3 – Modifier shell, nom du compte et répertoire personnel

```
sudo usermod -s /bin/bash stagiaire1            # changer le shell
sudo usermod -l stagiaire2 stagiaire1            # renommer le compte
sudo usermod -d /home/stagiaire2 -m stagiaire2   # déplacer le home
```

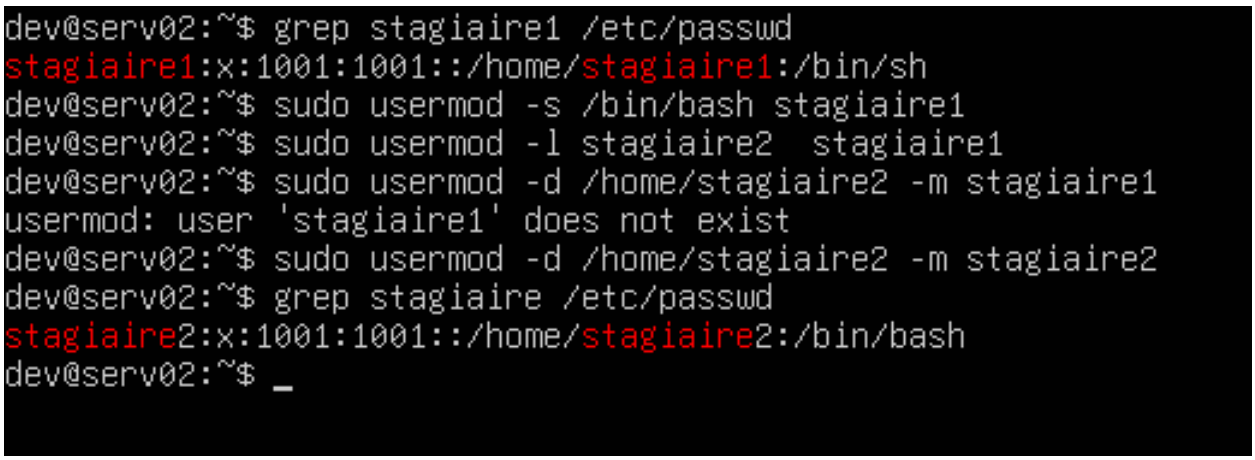


Figure 12 – Modifications successives du compte ; l'erreur « user 'stagiaire1' does not exist » apparaît puis est corrigée.

DIFFICULTÉ RENCONTRÉE ET SOLUTION

Après le renommage avec `usermod -l`, l'ancien nom n'existe plus. Ma commande `usermod -d /home/stagiaire2 -m stagiaire1` a donc échoué. J'ai compris que **l'ordre des opérations compte** : une fois le compte renommé, toutes les commandes suivantes doivent utiliser le nouveau nom. J'ai relancé la commande avec `stagiaire2` et elle a fonctionné.

POINT D'ATTENTION : LE GROUPE PRINCIPAL

Le renommage d'un compte ne renomme ni son répertoire personnel ni son groupe principal : ce sont des éléments distincts. Les captures suivantes (commande `groups`) montrent que le groupe principal s'appelle encore **stagiaire1**. La correction s'effectue avec `sudo groupmod -n stagiaire2 stagiaire1`.

Étape 4 – Gérer l'expiration du compte et du mot de passe

La commande `chage -l` affiche la politique d'expiration. Sans `sudo`, elle est refusée ; sur l'ancien nom, elle signale que le compte n'existe plus.

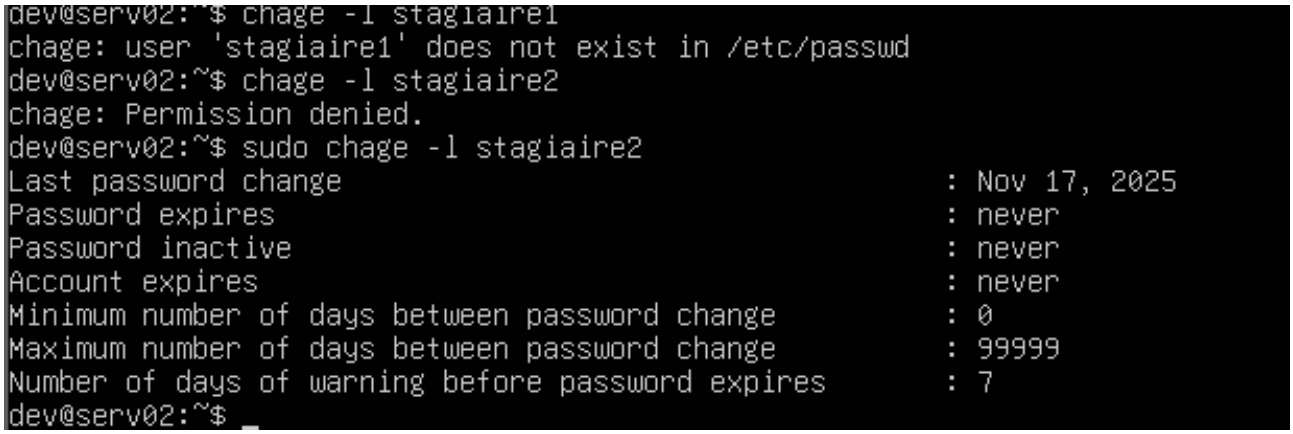


Figure 13 – chage -l : erreur sur l'ancien nom, accès refusé sans sudo, puis consultation réussie avec sudo.

```
dev@serv02:~$ sudo chage -l stagiaire2
Last password change                                    : Nov 17, 2025
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
dev@serv02:~$ sudo chage
Usage: chage [options] LOGIN

Options:
  -d, --lastday LAST_DAY        set date of last password change to LAST_DAY
  -E, --expiredate EXPIRE_DATE  set account expiration date to EXPIRE_DATE
  -h, --help                    display this help message and exit
  -i, --iso8601                 use YYYY-MM-DD when printing dates
  -I, --inactive INACTIVE       set password inactive after expiration
                                to INACTIVE
  -l, --list                    show account aging information
  -m, --mindays MIN_DAYS        set minimum number of days before password
                                change to MIN_DAYS
  -M, --maxdays MAX_DAYS        set maximum number of days before password
                                change to MAX_DAYS
  -R, --root CHROOT_DIR         directory to chroot into
  -W, --warndays WARN_DAYS      set expiration warning days to WARN_DAYS

dev@serv02:~$ _
```

Figure 14 – Résultat de chage -l et liste des options de la commande chage.

Le compte et son mot de passe n'expirent jamais (valeur « never », durée maximale 99999 jours), avec un avertissement 7 jours avant l'expiration. Les options affichées (`-E` date d'expiration du compte, `-M` durée maximale du mot de passe, `-W` jours d'avertissement, `-I` délai d'inactivité) permettent de durcir cette politique.

> **POUR ALLER PLUS LOIN (NON EXÉCUTÉ DANS CE TP)**
> Dans ce TP, la politique a été consultée mais non modifiée. Exemple de durcissement pour un compte de stagiaire : `sudo chage -M 90 -W 7 -E 2026-01-31 stagiaire2` (mot de passe valable 90 jours, compte expirant à la fin du stage).

Étape 5 – Verrouiller et déverrouiller le compte

```
sudo passwd -l stagiaire2    # verrouiller
sudo passwd -S stagiaire2    # afficher le statut
sudo passwd -u stagiaire2    # déverrouiller
```

```
stagiaire2:x:1001:1001::/home/stagiaire2:/bin/b
dev@serv02:~$ sudo passwd -l stagiaire2
[sudo] password for dev:
passwd: password changed.
dev@serv02:~$
dev@serv02:~$ sudo passwd -S stagiaire2
stagiaire2 L 2025-11-17 0 99999 7 -1
```

Figure 15 – Verrouillage : le statut passe à « L » (locked).

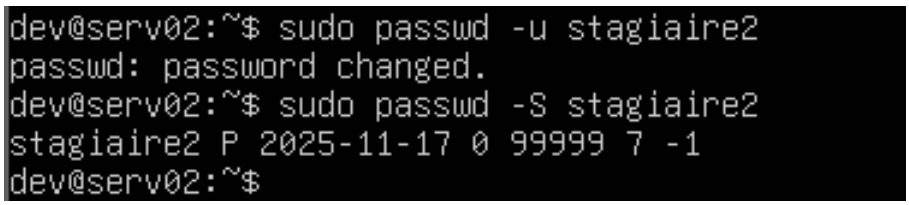


Figure 16 – Déverrouillage : le statut revient à « P » (mot de passe utilisable).

Le verrouillage place un « ! » devant le hash dans `/etc/shadow` : le mot de passe n'est plus valide, mais le compte et ses fichiers sont conservés. C'est la bonne réaction en cas de départ ou de compromission suspectée, avant toute suppression.

Étape 6 – Supprimer un compte en conservant son home

> **ÉTAPE NON EXÉCUTÉE DANS CE TP**
> Le sujet demandait `userdel stagiaire1`. Je n'ai pas supprimé le compte afin de le réutiliser dans les parties suivantes (groupes et sudo). Principe : `userdel` seul supprime le compte et **conserve** le répertoire personnel, tandis que `userdel -r` supprime aussi le home. Conserver le home permet de récupérer le travail d'un stagiaire parti.

> **RÉSULTAT OBTENU**
> Un compte stagiaire est créé, vérifié, modifié (shell, nom, répertoire), verrouillé puis déverrouillé. J'ai compris la différence entre compte, répertoire personnel et groupe principal, qui sont trois objets liés mais indépendants.

PARTIE4 :

Gestion des groupes et organisation du système

Objectif : regrouper les utilisateurs pour attribuer les droits une seule fois au groupe plutôt que compte par compte.

Étape 1 – Créer un groupe et y ajouter un utilisateur

```
sudo groupadd devs
sudo usermod -aG devs stagiaire2
```

L'option `-a` (append) est essentielle : sans elle, `usermod -G` **remplace** tous les groupes secondaires du compte au lieu d'en ajouter un. Comme stagiaire1 avait été renommé, c'est stagiaire2 qui a été ajouté à devs.

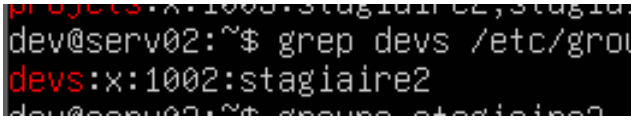


Figure 17 – Groupe devs (GID 1002) contenant stagiaire2.

Étape 2 – Vérifier l'appartenance aux groupes

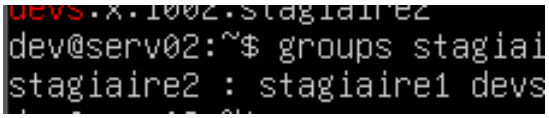


Figure 18 – Commande groups : stagiaire2 appartient à stagiaire1 (groupe principal) et à devs.

Étape 3 – Créer le groupe projets avec plusieurs membres

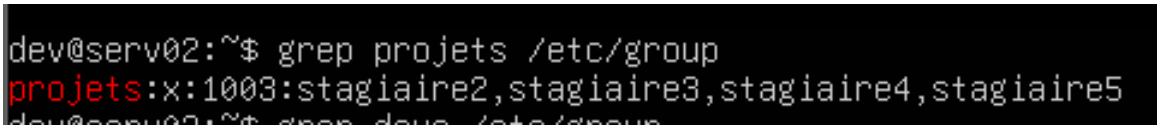


Figure 19 – Groupe projets (GID 1003) avec stagiaire2, stagiaire3, stagiaire4 et stagiaire5.

Les comptes stagiaire3 à stagiaire5 existent bien (leur présence est confirmée par la commande `groups` plus bas).

Étape 4 – Renommer un groupe et modifier son GID

```
sudo groupmod -n projets_dev projets   # renommer
sudo groupmod -g 2000 projets_dev       # changer le GID
```

```
dev@serv02:~$ sudo groupmod -n projets_dev projets
dev@serv02:~$ grep projets_dev /etc/group
projets_dev:x:1003:stagiaire2,stagiaire3,stagiaire4,stagiaire5
dev@serv02:~$ sudo groupmod -g 2000 projets_dev
dev@serv02:~$ grep projets_dev /etc/group
projets_dev:x:2000:stagiaire2,stagiaire3,stagiaire4,stagiaire5
dev@serv02:~$ groups dev
dev : dev adm cdrom sudo dip plugdev lxd
dev@serv02:~$ groups stagiaire2
stagiaire2 : stagiaire1 devs projets_dev
dev@serv02:~$ groups stagiaire3
stagiaire3 : stagiaire3 projets_dev
dev@serv02:~$
```

Figure 20 – Renommage en projets_dev puis changement du GID de 1003 à 2000, observés dans /etc/group.

À RETENIR

Changer le GID d'un groupe est une opération sensible : les fichiers qui portaient l'ancien GID deviennent « orphelins » (leur groupe s'affiche sous forme de numéro). Ici, aucun fichier n'appartenait encore au groupe, donc l'opération est sans conséquence. En production, il faudrait ensuite réaffecter les fichiers avec `chgrp -R`.

Étape 5 – Supprimer un groupe et analyser les conséquences

```
sudo groupdel devs
grep devs /etc/group          # plus de résultat
groups stagiaire2
```

```
dev@serv02:~$ sudo groupdel devs
dev@serv02:~$ grep devs /etc/group
dev@serv02:~$ groups stagiaire2
stagiaire2 : stagiaire1 projets_dev
dev@serv02:~$ sudo groupdel stagiaire2
groupdel: group 'stagiaire2' does not exist
dev@serv02:~$ ls -l devs/stagiaire2
ls: cannot access 'devs/stagiaire2': No such file or directory
dev@serv02:~$ ls -l home/stagiaire2
ls: cannot access 'home/stagiaire2': No such file or directory
dev@serv02:~$ ls -l /home/stagiaire2
ls: cannot open directory '/home/stagiaire2': Permission denied
dev@serv02:~$ sudo ls -l /home/stagiaire2
total 0
```

Figure 21 – Suppression du groupe devs et vérifications : devs disparaît de /etc/group et de la liste des groupes de stagiaire2.

Analyse des conséquences :

- Les utilisateurs perdent l'appartenance au groupe supprimé (visible avec `groups stagiaire2`).
- Les fichiers et dossiers qui appartenaient à ce groupe affichent un GID numérique orphelin.

- Si des droits dépendaient de ce groupe, les membres n'y ont plus accès via ce groupe.
- La tentative `groupdel stagiaire2` a échoué (« group does not exist ») : cela confirme que le groupe principal s'appelle encore stagiaire1.
- Le répertoire `/home/stagiaire2` est refusé à dev sans sudo : un home n'est lisible que par son propriétaire (et son groupe).

Étape 6 – Dossier commun /srv/dev réservé au groupe

```
sudo mkdir -p /srv/dev
sudo chown :projets_dev /srv/dev   # groupe propriétaire
sudo chmod 770 /srv/dev            # rwx pour propriétaire et groupe, rien pour les autres
sudo chmod g+s /srv/dev            # SetGID : les nouveaux fichiers héritent du groupe
```

```
dev@serv02:~$ groups stagiaire2
stagiaire2 : stagiaire1 projets_dev
dev@serv02:~$ mkdir -p /srv/dev
mkdir: cannot create directory ‘/srv/dev’: Permission denied
dev@serv02:~$ sudo mkdir -p /srv/dev
dev@serv02:~$ sudo chown :devs /srv/dev
chown: invalid group: ‘:devs’
dev@serv02:~$ sudo chown :projets_dev /srv/dev
dev@serv02:~$ sudo chmod 770 /srv/dev
dev@serv02:~$ sudo chmod g+s /srv/dev
dev@serv02:~$
```

Figure 22 – Création de /srv/dev : refus sans sudo, erreur « invalid group: devs », puis réussite avec projets_dev.

DIFFICULTÉ RENCONTRÉE ET SOLUTION

Ma commande `sudo chown :devs /srv/dev` a échoué avec « invalid group » : j'avais supprimé le groupe devs à l'étape précédente sans le recréer. J'ai adapté la suite du TP en utilisant le groupe **projets_dev**, qui remplit le même rôle. Cette erreur illustre concrètement les conséquences d'une suppression de groupe.

RÉSULTAT OBTENU

Le dossier `/srv/dev` appartient au groupe projets_dev et n'est accessible qu'à son propriétaire et aux membres du groupe (mode 770 + SetGID, soit 2770). Les autres utilisateurs n'ont aucun droit. Le contrôle avec `ls -ld /srv/dev` n'a pas été capturé ; le même mode a été vérifié sur `/home/projet` dans la partie suivante.

PARTIE 5 :

Permissions, modes spéciaux et sécurité

Objectif : maîtriser les permissions Linux et comprendre à quoi servent les trois modes spéciaux, qui sont des points sensibles d'un audit de sécurité.

Rappel : chaque fichier a trois jeux de droits (propriétaire, groupe, autres) composés de **r** (lire, valeur 4), **w** (écrire, 2) et **x** (exécuter, 1). Ainsi `750` signifie rwx pour le propriétaire (4+2+1), r-x pour le groupe (4+1) et aucun droit pour les autres.

Étape 1 – Créer le dossier /home/projet et le fichier notes.txt

```
chown: invalid group: ':devs'
dev@serv02:~$ sudo chown :projets_dev /srv/dev
dev@serv02:~$ sudo chmod 770 /srv/dev
dev@serv02:~$ sudo chmod g+s /srv/dev
dev@serv02:~$ sudo mkdir /home/projet
dev@serv02:~$ sudo touch /home/projet/notes.txt
dev@serv02:~$
```

Figure 23 – Création de /home/projet et du fichier notes.txt avec sudo.

Étape 2 – Obtenir les droits rwxr-x---

```
sudo chmod 750 /home/projet
sudo chmod 750 /home/projet/notes.txt
sudo ls -l /home/projet
```

```
dev@serv02:~$ sudo mkdir /home/projet
dev@serv02:~$ sudo touch /home/projet/notes.txt
dev@serv02:~$ sudo chmod 750 /home/projet
dev@serv02:~$ sudo chmod 750 /home/projet/notes.txt
dev@serv02:~$ ls -l /home/projet
ls: cannot open directory '/home/projet': Permission denied
dev@serv02:~$ sudo ls -l /home/projet
total 0
-rwxr-x--- 1 root root 0 Nov 17 11:04 notes.txt
dev@serv02:~$
```

Figure 24 – Droits 750 appliqués : le fichier notes.txt affiche -rwxr-x---.

> **OBSERVATION**
> Sans sudo, `ls -l /home/projet` renvoie « Permission denied » : dev n'est ni propriétaire (root) ni membre du groupe, il fait donc partie des « autres », qui n'ont aucun droit. C'est le comportement attendu et la preuve que les droits fonctionnent. À noter : le bit x sur un simple fichier texte n'est pas utile, il était ici imposé par le sujet.

Étape 3 – Changer le groupe propriétaire

```
sudo chgrp projets_dev /home/projet
```

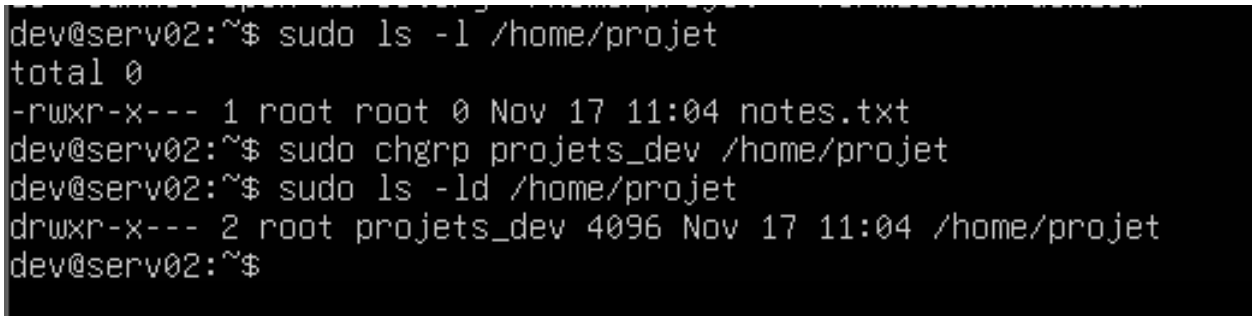


Figure 25 – Le dossier /home/projet appartient maintenant au groupe projets_dev (drwxr-x---).

Étape 4 – Activer le SetGID sur le dossier

```
sudo chmod 2770 /home/projet
```

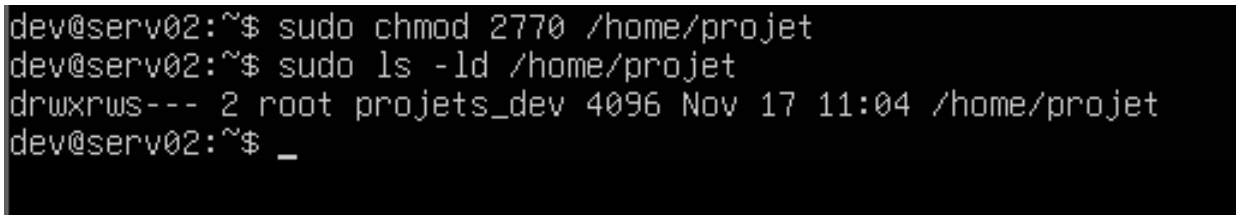


Figure 26 – Mode drwxrws--- : le « s » à la place du x du groupe indique que le SetGID est actif.

Effet du SetGID sur un dossier : quand un membre du groupe projets_dev crée un fichier ou un sous-dossier dans `/home/projet`, le nouvel élément reçoit automatiquement **projets_dev** comme groupe propriétaire, même si l'utilisateur appartient à plusieurs groupes. Tous les membres peuvent ainsi travailler sur les mêmes fichiers sans manipulation.

> **REMARQUE**
> Le SetGID n'agit que sur les **nouveaux** éléments. Le fichier notes.txt, créé avant, garde son groupe d'origine (root) tant qu'on ne le change pas avec `chgrp`.

Étape 5 – Sticky Bit sur un dossier partagé

```
sudo mkdir /tmp/partage
sudo chmod 1777 /tmp/partage
sudo ls -ld /tmp/partage
```

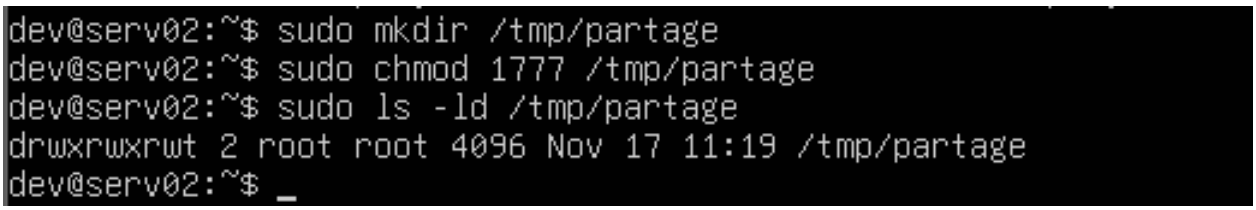


Figure 27 – Mode drwxrwxrwt : le « t » final indique le Sticky Bit.

Le **Sticky Bit** empêche un utilisateur de supprimer ou renommer les fichiers des autres dans un dossier, même si tout le monde y a le droit d'écriture. C'est le mécanisme utilisé par le dossier système `/tmp`.

Étape 6 – SetUID sur un script

```
sudo nano /usr/local/bin/helloroot.sh
sudo chmod 755 /usr/local/bin/helloroot.sh
sudo chmod u+s /usr/local/bin/helloroot.sh
ls -l /usr/local/bin/helloroot.sh
```

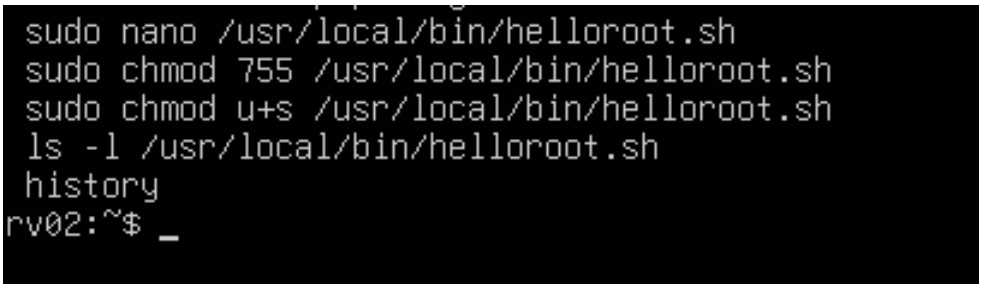


Figure 28 – Historique des commandes : création du script, chmod 755 puis chmod u+s.

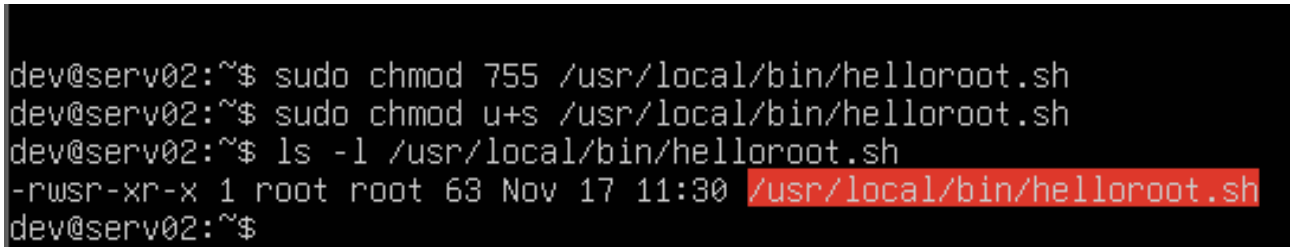


Figure 29 – Mode -rwsr-xr-x : le « s » à la place du x du propriétaire indique le SetUID (propriétaire root).

Le **SetUID** fait qu'un programme s'exécute avec les droits de son **propriétaire** (souvent root) et non avec ceux de l'utilisateur qui le lance. Il est sensible car une faille dans un programme SetUID root peut permettre à un utilisateur normal d'obtenir des droits d'administrateur.

POINT DE SÉCURITÉ

Sur Linux, le noyau ignore le SetUID sur les scripts interprétés (shell, Python) : le bit s'affiche mais n'a pas d'effet. Le vrai risque concerne les **binaires compilés**. Un audit de sécurité commence par les lister : `find / -perm -4000 -type f 2>/dev/null`. Tout binaire SetUID inattendu doit être examiné.

Mode spécial	Valeur	Affichage	Effet	Cas d'usage
SetUID	4	s (propriétaire)	Exécute avec les droits du propriétaire	passwd, sudo
SetGID	2	s (groupe)	Sur un dossier : les nouveaux fichiers héritent du groupe	Dossier d'équipe /srv/dev
Sticky Bit	1	t (autres)	Seul le propriétaire peut supprimer son fichier	/tmp, /tmp/partage

RÉSULTAT OBTENU

Droits 750 et 2770 appliqués et vérifiés, Sticky Bit actif sur /tmp/partage, SetUID posé sur un script et son risque expliqué. Je sais lire un mode en octal comme en symbolique (rwx, s, t).

PARTIE 6 :

Gestion des privilèges avec sudo et traçabilité

Objectif : déléguer des droits d'administration précis sans partager le mot de passe root, puis vérifier que chaque action est tracée.

Étape 1 – Écrire une règle sudoers avec visudo

Le fichier `/etc/sudoers` se modifie uniquement avec `visudo`, qui verrouille le fichier et contrôle la syntaxe avant d'enregistrer. Une erreur de syntaxe enregistrée directement pourrait bloquer l'accès à sudo pour tout le monde.

```
sudo visudo

# Ligne ajoutée à la fin du fichier :
stagiaire1 ALL=(ALL:ALL) /sbin/reboot, /usr/bin/apt update
```

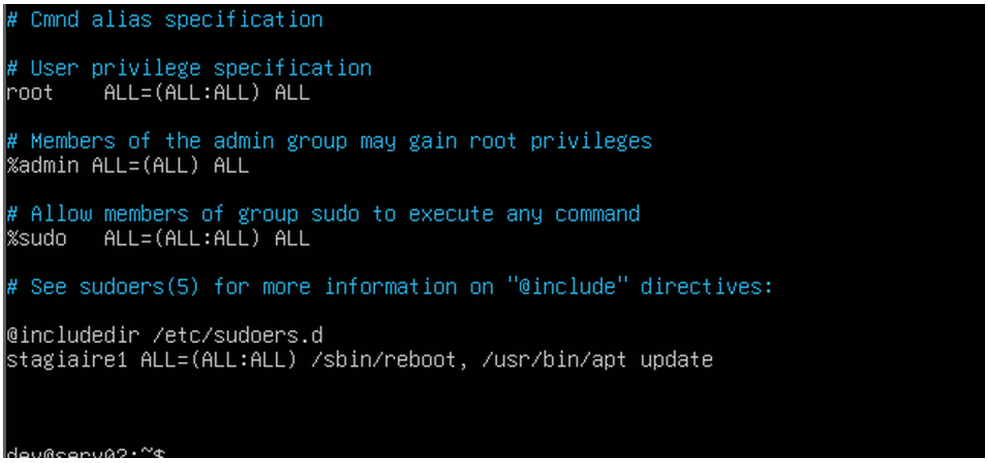


Figure 30 – Fin du fichier sudoers (extrait agrandi) : règle limitée à reboot et apt update.

La syntaxe se lit ainsi : *utilisateur machine* = (*utilisateur cible*:*groupe cible*) *commandes autorisées*. La règle n'autorise donc que deux commandes précises, ce qui respecte le principe du moindre privilège.

> **POINT D'ATTENTION IDENTIFIÉ À LA RELECTURE**
> Cette règle désigne **stagiaire1**, or ce compte a été renommé stagiaire2 : la règle ne s'applique donc plus à personne. Il faut remplacer le nom par stagiaire2, ou mieux, lier la règle à un groupe. On peut contrôler les droits d'un compte avec `sudo -l -U stagiaire2`.

Étape 2 – Tester une commande nécessitant les privilèges root

Ce test a été lancé par le compte **dev** (membre de sudo), comme le confirme le journal d'authentification. Il valide le fonctionnement de sudo, mais pas la délégation au stagiaire : pour cela, il faudrait ouvrir une session avec le compte concerné.

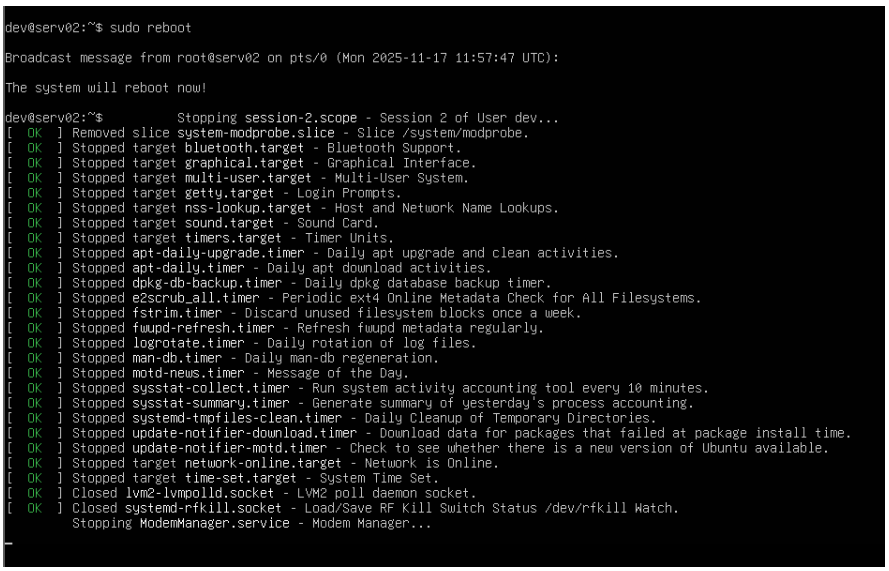


Figure 31 – sudo reboot : le système annonce son redémarrage (message de diffusion et arrêt des services).

Étape 3 – Créer le groupe adminlocal avec des droits sudo personnalisés

```
sudo groupadd adminlocal
sudo usermod -aG adminlocal stagiaire2
sudo visudo

# Ligne ajoutée :
%adminlocal ALL=(ALL:ALL) /usr/bin/systemctl restart apache2
```

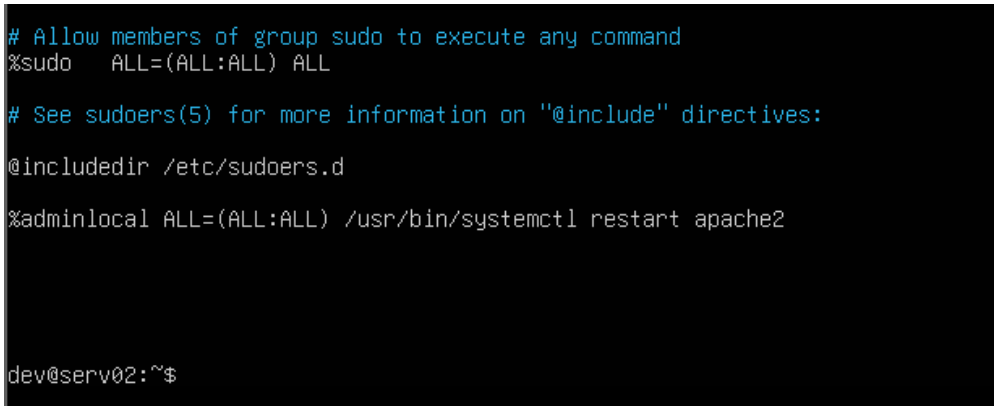


Figure 32 – Règle de groupe : les membres de adminlocal peuvent seulement redémarrer apache2.

Le symbole **%** désigne un groupe. Un membre de adminlocal peut redémarrer le serveur web, par exemple après une mise à jour de site, mais ne peut rien faire d'autre avec les droits root. Attribuer les droits à un groupe est plus simple à maintenir : on ajoute ou retire une personne sans modifier le fichier sudoers.

Étape 4 – Retirer les droits sudo

```
sudo gpasswd -d stagiaire2 adminlocal
```

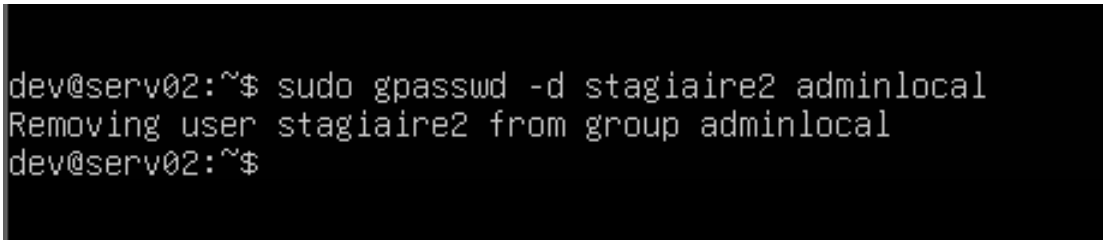


Figure 33 – Retrait de stagiaire2 du groupe adminlocal.

Le retrait du groupe suffit à supprimer la délégation, sans toucher à la règle. **Impact attendu :** à sa prochaine connexion, stagiaire2 ne pourra plus lancer la commande autorisée via sudo et recevra un message de refus. Ce test n'a pas été capturé ; il reste à réaliser avec une session du compte concerné.

Étape 5 – Analyser la traçabilité dans /var/log/auth.log

```
sudo grep sudo /var/log/auth.log
```

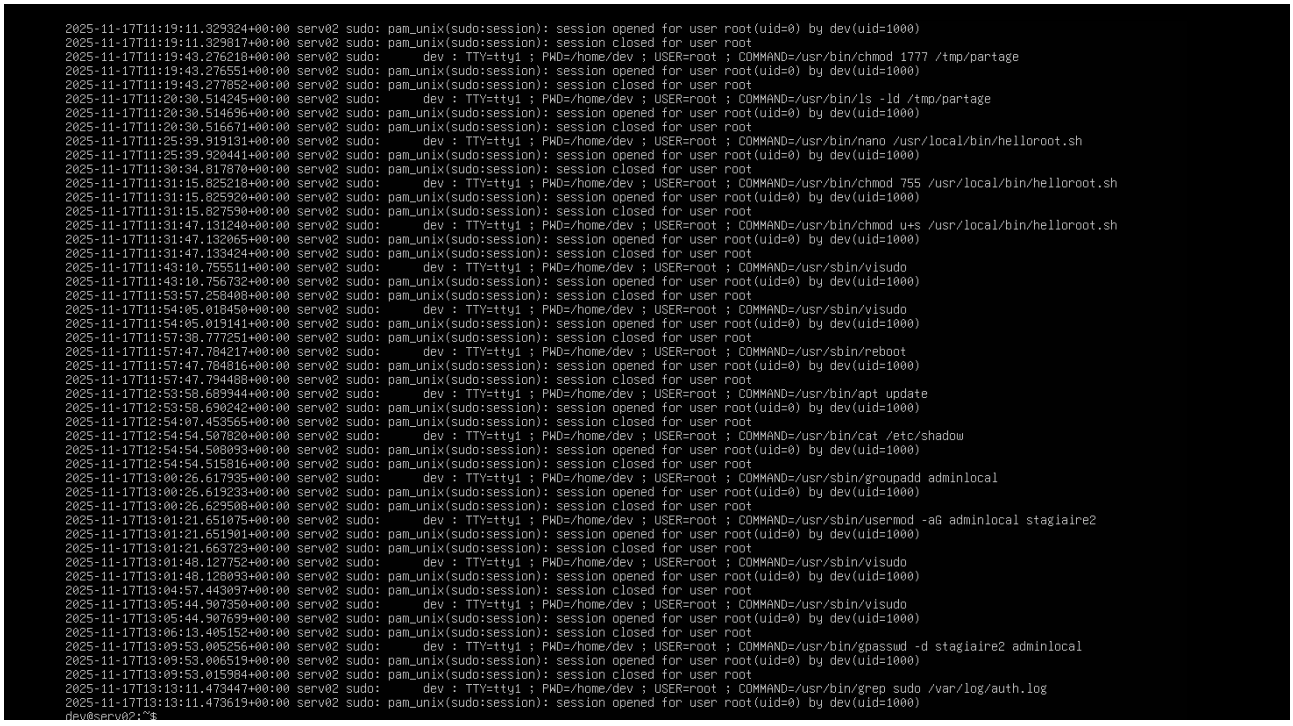


Figure 34 – Journal /var/log/auth.log : toutes les utilisations de sudo, avec heure, utilisateur et commande.

Chaque ligne indique **qui** (utilisateur), **quand** (horodatage, en UTC), **depuis où** (terminal) et **quoi** (commande complète). Extrait des commandes administratives de ce projet :

Heure (UTC)	Commande tracée (exécutée en tant que root par dev)	Signification
11:57:47	`/usr/sbin/reboot`	Test d'exécution de commande root
12:54:54	`/usr/bin/cat /etc/shadow`	Lecture d'un fichier sensible
13:00:26	`/usr/sbin/groupadd adminlocal`	Création du groupe de délégation
13:01:21	`/usr/sbin/usermod -aG adminlocal stagiaire2`	Ajout d'un membre au groupe
13:01:48 / 13:05:44	`/usr/sbin/visudo`	Modifications du fichier sudoers
13:09:53	`/usr/bin/gpasswd -d stagiaire2 adminlocal`	Retrait des droits délégués
13:13:11	`/usr/bin/grep sudo /var/log/auth.log`	Consultation du journal lui-même

À RETENIR SUR LA TRAÇABILITÉ

Même l'accès au fichier `/etc/shadow` est enregistré : un administrateur ne peut pas agir sur le système sans laisser de trace. C'est un avantage décisif de sudo par rapport au partage du mot de passe root, où l'on ne saurait pas qui a fait quoi.

RÉSULTAT OBTENU

Deux délégations de droits sont écrites (compte, puis groupe adminlocal), le retrait d'un droit est réalisé, et la traçabilité est démontrée dans le journal.

PARTIE 7 :

Tests, résultats et bilan du projet

1) Tableau de validation des tests

Test	Vérification	Résultat observé	Statut
Lecture des fichiers d'identité	`cat /etc/passwd /etc/group`	Contenus affichés et analysés	Validé
Protection de shadow	`cat /etc/shadow`	Refus sans sudo, accès avec sudo	Validé
Création du compte et du home	`ls /home`	Dossier /home/stagiaire1 présent	Validé
Modification shell, nom, home	`grep stagiaire /etc/passwd`	stagiaire2 avec /home/stagiaire2 et /bin/bash	Validé
Renommage du groupe principal	`groups stagiaire2`	Groupe principal toujours nommé stagiaire1	Partiel
Politique d'expiration	`sudo chage -l`	Consultée ; aucun changement appliqué	Partiel
Verrouillage / déverrouillage	`passwd -S`	Statut L puis P	Validé
Suppression du compte (home conservé)	`userdel`	Non exécutée (compte réutilisé)	Non réalisé
Groupes et membres	`grep ... /etc/group`	devs et projets créés avec membres	Validé
Renommage et changement de GID	`grep projets_dev /etc/group`	projets_dev, GID 2000	Validé
Suppression d'un groupe	`groupdel devs`	Conséquences observées (chown refusé)	Validé
Dossier de groupe /srv/dev	`chown / chmod 770 / g+s`	Commandes acceptées ; ls -ld non capturé	Partiel
Droits 750	`ls -l /home/projet`	-rwxr-x---	Validé
SetGID	`ls -ld /home/projet`	drwxrws---	Validé
Sticky Bit	`ls -ld /tmp/partage`	drwxrwxrwt	Validé
SetUID	`ls -l helloroot.sh`	-rwsr-xr-x ; effet non testé	Partiel
Règle sudoers sur un compte	`sudo reboot`	Testé par dev ; règle sur stagiaire1 (renommé)	Partiel
Groupe adminlocal	`visudo`	Règle %adminlocal écrite	Validé
Retrait des droits	`gpasswd -d`	Retrait réalisé ; refus non capturé	Partiel
Traçabilité	`grep sudo auth.log`	Actions sudo horodatées et attribuées	Validé

2) Résultats obtenus

Le serveur serv02 dispose désormais d'une organisation claire : un compte stagiaire paramétré, des groupes de travail, un dossier de groupe protégé, des modes spéciaux compris et vérifiés, et une délégation sudo limitée à des commandes précises dont l'historique est conservé. Les résultats sont démontrés par les captures de chaque partie. Les tests marqués « Partiel » ou « Non réalisé » sont signalés avec honnêteté : ils sont identifiés et corrigeables, ce qui fait aussi partie d'une démarche professionnelle.

3) Difficultés rencontrées et solutions

Difficulté	Cause	Solution et enseignement
« command not found » sur sudo cat/etc/shadow	Espace manquant entre la commande et son argument	Corriger la syntaxe ; relire sa commande avant de valider
« user 'stagiaire1' does not exist »	Compte déjà renommé avec usermod -l	Utiliser le nouveau nom ; l'ordre des opérations compte
Groupe principal resté stagiaire1	Compte, home et groupe sont trois objets distincts	groupmod -n pour renommer le groupe
« invalid group: devs » avec chown	Groupe supprimé à l'étape précédente	Utiliser projets_dev ; mesurer l'impact d'un groupdel
« Permission denied » sur /home/projet	Dossier en 750 appartenant à root	Comportement normal : utiliser sudo ou un membre du groupe
Règle sudoers sans effet pour stagiaire2	Règle écrite pour l'ancien nom du compte	Corriger le nom ou passer par un groupe ; tester avec le compte concerné

4) Compétences mobilisées

Compétence (référentiel BTS SIO SISR)	Mise en œuvre dans ce projet	Preuves
B1.5 – Mettre à disposition des utilisateurs un service informatique	Création et paramétrage des comptes stagiaires, espace de travail partagé	Parties 3 et 4
B2.2 – Installer, tester et déployer une solution d'infrastructure	Configuration des comptes, groupes et droits ; tests de vérification	Parties 3 à 6
B2.3 – Exploiter, dépanner et superviser	Diagnostic des erreurs (does not exist, invalid group, permission denied) ; lecture des journaux	Parties 3 à 6
B3.3 – Sécuriser les équipements et les usages des utilisateurs	Verrouillage de compte, droits restreints, Sticky Bit, moindre privilège	Parties 3, 5, 6
B3.4 – Garantir la disponibilité, l'intégrité et la confidentialité des données	Protection de /etc/shadow, dossier réservé à un groupe, traçabilité	Parties 2, 4, 6
B3.5 – Assurer la cybersécurité d'un système	Analyse du SetUID, sudoers via visudo, analyse de auth.log	Parties 5 et 6
Compétences transversales	Documenter, analyser ses erreurs, présenter un résultat à l'écrit	Tout le document

5) Axes d'amélioration

- Corriger la règle sudoers pour stagiaire2 (ou la lier à un groupe) et la tester avec une session du compte concerné.
- Renommer le groupe principal avec `groupmod -n stagiaire2 stagiaire1`.
- Appliquer une politique d'expiration avec `chage` pour les comptes temporaires.
- Ranger les règles dans un fichier dédié avec `visudo -f /etc/sudoers.d/adminlocal` plutôt que dans le fichier principal.
- Auditer régulièrement les binaires SetUID et les membres des groupes sensibles (sudo, lxd).
- Appliquer à chaque compte un mot de passe robuste, imposé par une politique de complexité.

6) Conclusion

Ce projet m'a permis de passer de la simple connaissance des commandes à une démarche d'administratrice : organiser les accès, limiter les droits au nécessaire, documenter chaque action et vérifier que tout est tracé. J'en retiens que le compte, son répertoire personnel et son groupe principal sont trois éléments distincts, que la suppression d'un groupe a des conséquences immédiates sur les accès, que les modes spéciaux sont puissants mais exigent de la rigueur, et que sudo associé à visudo offre à la fois un contrôle fin et une traçabilité complète.

La suite logique de ce travail est l'intégration à un annuaire centralisé (Active Directory ou LDAP), qui applique les mêmes principes de comptes, de groupes et de moindre privilège à l'échelle de toute une organisation, et la supervision des journaux pour détecter les usages anormaux.

ANNEXE

Récapitulatif des commandes utilisées

Commande	Rôle
`cat, less, tail`	Afficher le contenu d'un fichier
`id / id -Gn / groups`	Afficher UID, GID et groupes d'un utilisateur
`getent passwd | group`	Interroger les comptes et groupes (fichiers ou annuaire)
`useradd -m`	Créer un compte avec son répertoire personnel
`passwd / passwd -l / -u / -S`	Définir le mot de passe, verrouiller, déverrouiller, afficher le statut
`usermod -s / -l / -d -m / -aG`	Changer shell, nom, répertoire personnel ; ajouter à un groupe
`chage -l / -M / -E / -W`	Consulter ou configurer l'expiration du mot de passe et du compte
`userdel [-r]`	Supprimer un compte (avec -r : home supprimé aussi)
`groupadd / groupmod -n -g / groupdel`	Créer, renommer, changer le GID, supprimer un groupe
`gpasswd -d`	Retirer un utilisateur d'un groupe
`chmod / chown / chgrp`	Modifier les droits, le propriétaire, le groupe
`chmod u+s / g+s / +t`	Activer SetUID, SetGID, Sticky Bit
`visudo`	Modifier sudoers avec contrôle de syntaxe
`sudo -l -U <compte>`	Lister les droits sudo d'un compte
`grep sudo /var/log/auth.log`	Rechercher les traces d'utilisation de sudo